バスケット構造対応 Apriori-Window による偽共起パターンの排除

（執筆者未定）

ドラフト—日本語版

***Abstract*—既存の頻出パターンマイニングは主にグローバル頻度に依存するため，特定期間に集中して現れる局所的な密集パターンを十分に捉えにくい．また，Gap 系手法は最大ギャップ閾値への感度が高く，パラメータ設定に結果が左右される．この課題に対して DSAA2025 では，局所的な出現頻度の変動を厳密に捉える Apriori-window が提案され，最大ギャップ閾値に依存しない密集パターン抽出を実現した．しかし同手法は，1 トランザクションを 1 アイテムセットとして扱う前提を置いており，同一時刻に複数の独立した購買イベント（バスケット）が含まれるデータでは，バスケット間のアイテムが誤って共起として数えられる偽共起が発生する．本稿はこの未解決課題に対して，DSAA2025 の枠組みを維持したままバスケット構造を扱える拡張手法（Basket-aware Apriori-Window, Phase 1）を提案する．提案手法は，共起判定をトランザクション内からバスケット内へ置き換えることで，偽共起パターン率（SPR: Spurious Pattern Rate）を原理的に 0 に抑える．さらに，バスケット内共起においても Apriori の反単調性が成立することを示し，従来と同様の候補枝刈りが可能であることを理論的に保証する．合成データ実験では，バスケット数パラメータ λ_{baskets} の増加に伴い従来法の過検出が急増し（A1: observed spurious の平均 1.0 → 379.0），カテゴリ数の増加で構造的な偽共起率が低下する傾向（A2: GT SPR の平均 0.827 → 0.638）を確認した．さらにスケーラビリティ評価（A4）では，10^6 トランザクションで提案手法 95.5 秒，従来法 143.6 秒（1.50 倍）となり，大規模条件での優位性を示した．**

***Index Terms*—frequent pattern mining, spurious co-occurrence, basket structure, sliding window, dense itemset**

I. はじめに

取引データ分析は，小売・EC・医療など多様な領域で意思決定の基盤となっている．とくに，特定期間に集中して出現するアイテム集合 (密集パターン) の抽出は，キャンペーン効果，季節性，需要変動を捉えるうえで重要である．

DSAA2025 では，この課題に対して Apriori-window が提案された [1]．Apriori-window は，従来法のようにデータ全体のグローバル頻度だけを見るのではなく，スライディングウィンドウで局所頻度を評価することで，最大ギャップ閾値に依存しない厳密な密集パターン抽出を可能にした．この点は，Gap 系手法がパラメータに敏感であるという課題に対する実用上の重要な改善である．

一方で，DSAA2025 の問題設定には「1 トランザクション = 1 バスケット」という暗黙の前提がある．しかし実データでは，同一時刻・同一日付に複数の独立した購買イベントがまとめて記録され，1 トランザクション内に複数バスケットが存在する場合がある．この状況でトランザクション全体を 1 つのアイテム集合として扱うと，本来は別バスケットに属するアイテム同士が誤って共起と数えられ，偽共起（spurious co-occurrence）が発生する．

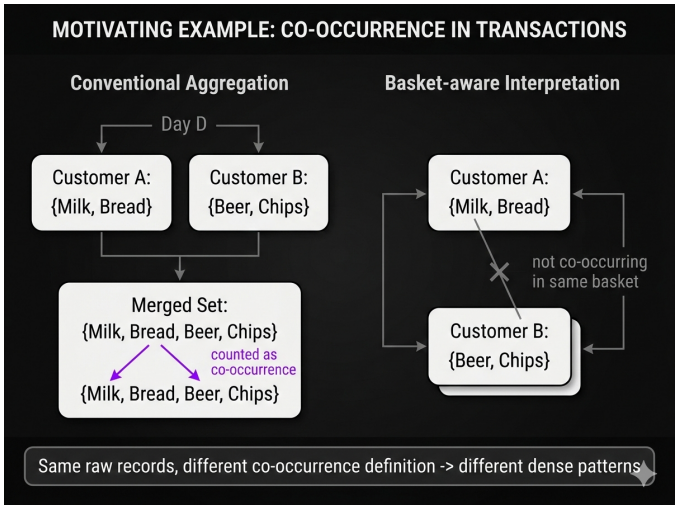

Fig. 1: 偽共起の動機付け例

例えば，同日に顧客 A が {牛乳, パン}，顧客 B が {ビール, ポテチ} を購入した場合を考える．日付単位で統合された 1 トランザクションをそのまま扱うと，(牛乳, ビール) のような実際には同一購買イベントで共起していない組合せが頻出として抽出され得る．この誤検出は，下流の需要予測や販促施策に誤った示唆を与える．

本研究の位置づけは，DSAA2025 の枠組みを否定することではなく，その有効性を維持したまま，未対応だった入力構造を拡張する点にある．具体的には，共起の定義を「トランザクション内」から「バスケット内」へ変更した Basket-aware Apriori-Window（Phase 1）を提案する．提案法の主貢献は以下の通りである．

- **課題の明確化**: DSAA2025 の単一バスケット前提が，複数バスケット記録データにおいて偽共起を生む構造的要因であることを示す．
- **手法拡張**: Apriori-window の枠組みを保ったまま，バスケット内共起に基づく探索へ拡張し，偽共起パターン率（SPR: Spurious Pattern Rate）を原理的に 0 に抑える．
- **理論的保証**: バスケット内共起においても Apriori の反単調性が成立し，候補枝刈りによる効率的探索が維持されることを示す．
- **実証**: 合成データ実験で，バスケット数パラメータ

λ_{baskets} 増加時の偽共起拡大（A1），カテゴリ構造の変化に対する偽共起率の感度（A2），および大規模条件での計算時間特性（A4）を確認する．

以上より，本稿は DSAA2025 の問題設定を現実のデータ構造へ拡張する第一段階として，密集パターン抽出の信頼性向上に寄与する．

II. 関連研究

A. DSAA2025 前手法

密集パターン抽出は，区間候補が概ね $O(n^2)$，さらに各区間内のパターン空間が $2^{|\mathcal{I}|}$ となるため，潜在探索空間が非常に大きい．この難しさに対し，既存研究の多くは問題を縮退させる形で実用化してきた．典型的には，(i) 全体支持度への還元，(ii) gap 制約・周期制約への置換，(iii) closed/maximal など列挙対象の圧縮である．

1) Apriori: Apriori は反単調性に基づく候補生成・枝刈りを用いる古典的手法であり，頻出パターンマイニングの基盤を与えた [2]．ただし評価対象はデータ全体のグローバル支持度であり，局所密集区間やバスケット境界は直接扱わない．

2) FP-Growth: FP-Growth は FP-tree により候補生成を回避し，高速に頻出パターンを列挙する [3]．大規模データで有効だが，基本設定はグローバル頻度問題であり，局所密集区間の直接抽出は目的外である．

3) ECLAT: ECLAT は縦型表現（tidset）と集合積により支持度計算を行う [4]．効率的な探索が可能だが，単位は「1 トランザクション =1 アイテム集合」であり，バスケット粒度の共起定義とは一致しない．

4) LCM: LCM 系は頻出・極大・閉パターン列挙で高効率を示す探索系統である [5]．一方で，列挙対象を圧縮する方向の最適化が中心であり，局所密集区間そのものを主目的にはしない．

5) PFPM: PFPM は periodic frequent pattern を対象とし，出現間隔の周期性制約を導入する [6]．ただし，最大ギャップ等の周期パラメータに結果が左右されやすく，密集定義を gap/periodicity に置き換える縮退型の枠組みに属する．

6) PPFPM (GPF-growth): PPFPM は partial periodicity を導入し，PFPM より柔軟に周期性を評価する [7]．しかし評価軸は周期的出現であり，局所密集区間の厳密抽出とは目的が異なる．

7) LPFIM: LPFIM は局所頻出パターンを区間単位で扱う初期研究である [8]．一方で区間抽出は gap 系しきい値に依存し，パラメータ感度が残る．

8) LPPM: LPPM は local periodic pattern と区間検出を扱う実用的手法である [9]．ただし周期パラメータ（maxPer/maxSoPer/minDur）依存が強く，ウィンドウ支持度による厳密密集定義とは異なる．

B. DSAA2025 Apriori-window

上記の縮退方向に対し，DSAA2025 の Apriori-window は，固定幅スライディングウィンドウ上で局所 support を直接評価し，密集パターンを厳密に抽出する枠組みを与えた [1]．すなわち，gap/periodicity に問題を置き換えずに，局所密集そのものを一次問題として扱った点に意義がある．

ただし Apriori-window は入力を「1 トランザクション =1 アイテム集合」と仮定するため，同一トランザクション内に複数の独立バスケットが存在する状況では，バスケット横断の誤共起（偽共起）を抑制できない．この点が，本研究が解くべき未解決課題である．

C. 本研究の位置づけ

本研究は，Apriori-window の核である「局所密集の厳密抽出」を維持しながら，前手法群でも Apriori-window でも十分に扱えなかったバスケット構造由来の偽共起問題を対象化する．具体的には，共起判定をトランザクション内からバスケット内へ拡張し，DSAA2025 枠組みの上位互換として実運用データへの適用可能性を高める立場をとる．

III. 問題定義

本節では，バスケット構造付きトランザクションデータの形式的定義を与え，偽共起の概念および解くべき問題を定式化する．

A. 基本用語

記法: $\mathcal{I} = \{i_1, i_2, \ldots, i_m\}$ を有限のアイテム全集合とする．$\mathcal{I}$ の任意の部分集合 $P \subseteq \mathcal{I}$ をアイテムセットと呼び，$|P|$ をその長さとする．また，バスケット B は 1 回の独立した購買イベントを表すアイテムセット $B \subseteq \mathcal{I}$ である．

定義 1 (バスケット構造付きトランザクション)**.** バスケット構造付きトランザクション（*basket-structured transaction*）はタプル $d = (ts, \mathcal{B})$ であり，$ts \in \mathbb{Z}_{\geq 0}$ はタイムスタンプ，$\mathcal{B} = \{B_1, B_2, \ldots, B_k\}$（$k \geq 1$）はそのタイムスタンプに属するバスケットの集合である．

データ列: バスケット構造付きトランザクションデータ $D = [d_1, d_2, \ldots, d_N]$ は，タイムスタンプの昇順に並べられたトランザクション列である．$N = |D|$ をデータ長と呼ぶ．

注意 (従来トランザクションとの関係): 定義 1において $k = 1$，すなわち各トランザクションがちょうど 1 つのバスケットをもつ場合が従来の頻出パターンマイニングにおけるトランザクションである．本研究は $k \geq 1$ を許容することで，この従来設定を特殊ケースとして包含する．

B. 共起の定義と偽共起

定義 2 (バスケット内共起（真の共起）)**.** トランザクション $d = (ts, \mathcal{B})$ において，アイテムセット P が バスケット内共起（*intra-basket co-occurrence*）するとは，P のすべてのアイテムが 同一のバスケット $B \in \mathcal{B}$ に含まれること，すなわち $\exists B \in \mathcal{B} : P \subseteq B$ が成立することをいう．

定義 3 (トランザクション内共起 (従来法の共起))**.** トランザクション $d = (ts, \mathcal{B})$ において，アイテムセット P がトランザクション内共起(*intra-transaction co-occurrence*)するとは，P のすべてのアイテムが（バスケットを横断して）そのトランザクションのどこかに存在すること，すなわち $P \subseteq \bigcup_{B \in \mathcal{B}} B$ が成立することをいう．

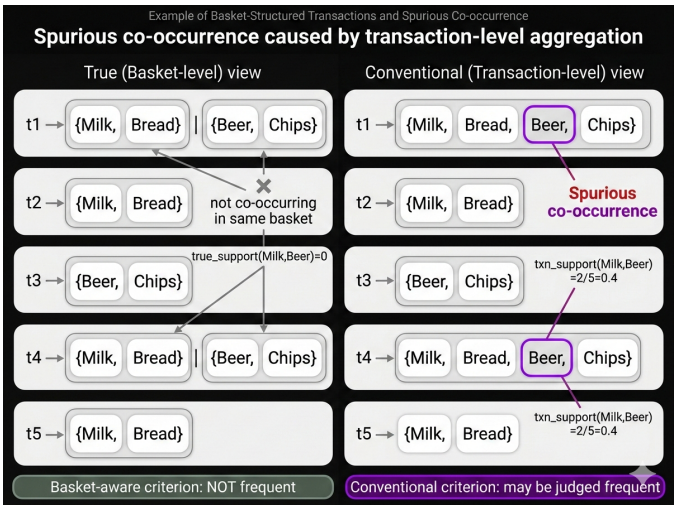

Fig. 2: バスケット構造付きトランザクションデータの例 (5 トランザクション)

定義 2 は定義 3 より真に強い条件である．すなわち，バスケット内共起 $\Rightarrow$ トランザクション内共起は常に成立するが，逆は一般に成立しない．異なるバスケット $B_j, B_k \in \mathcal{B}$（$j \neq k$）にそれぞれ属するアイテム同士は，トランザクション内共起はするものの，バスケット内共起はしない．

例 1 (偽共起の発生)**.** 図 *2* に示すデータを考える．この図では，同一日に複数バスケットが存在し得る状況を表現しており，バスケット内共起とトランザクション内共起の差が可視化されている．*5* つのトランザクション（日付）が存在し，各日に *1*〜*2* 件のバスケットが記録される．

t	バスケット *1*	バスケット *2*
1	{牛乳, パン}	{ビール, ポテチ}
2	{牛乳, バター}	—
3	{ビール, ポテチ}	{牛乳, チーズ}
4	{牛乳, パン}	{ビール, ポテチ}
5	{牛乳, パン}	—

アイテムセット P = {牛乳, ビール} について，$txn_support(P) = 3/5 = 0.60$（$t = 1, 3, 4$ でトランザクション内共起），$true_support(P) = 0/5 = 0.00$（どのバスケットにも同時に含まれない）．

最小サポート $\theta = 0.5$ に対して，従来法（トランザクション内共起）では P を頻出パターンとして抽出するが，これは実際には存在しない共起関係である．

次に，区間上のサポートを定式化する．

区間記法: 区間 $I = [ts_i, ts_j]$（$1 \leq i \leq j \leq N$）に対して，$D_I = [d_i, d_{i+1}, \ldots, d_j]$ を区間 I 内のトランザクション列とし，$n(D_I) = j - i + 1$ をその長さとする．

サポート関数: 区間 I におけるアイテムセット P のバスケット内サポート（true support）を

$$\mathrm{true_support}_I(P) = \frac{\left|\left\{d = (ts, \mathcal{B}) \in D_I \;\middle|\; \exists B \in \mathcal{B},\ P \subseteq B\right\}\right|}{n(D_I)} \tag{1}$$

とし，トランザクション内サポート（txn support）を

$$\mathrm{txn_support}_I(P) = \frac{\left|\left\{d = (ts, \mathcal{B}) \in D_I \;\middle|\; P \subseteq \bigcup_{B \in \mathcal{B}} B\right\}\right|}{n(D_I)} \tag{2}$$

とする．

命題 1 (単調性)**.** 任意のアイテムセット P，任意の区間 I，任意のトランザクションデータ D に対して，

$$true_support_I(P) \leq txn_support_I(P) \tag{3}$$

が成立する．

Proof. バスケット内共起は定義よりトランザクション内共起の十分条件であるため，P がバスケット内共起するトランザクションの集合は，P がトランザクション内共起するトランザクションの集合の部分集合である．したがって，式 (1) の分子 $\leq$ 式 (2) の分子であり，命題が成立する． □

C. 密集パターンと偽共起パターン

本稿では，先行研究 Apriori-window [1] の密集パターン定義をバスケット内サポートに基づいて再定義する．

定義 4 (バスケット対応密集区間・密集パターン)**.** 最小サポートしきい値 θ（$0 < \theta \leq 1$）および最小区間長 W（$W \in \mathbb{Z}_{\geq 1}$）を与える．区間 $I = [ts_i, ts_j]$（$n(D_I) \geq W$）がアイテムセット P の バスケット対応密集区間（*basket-aware dense interval*）であるとは，$true_support_I(P) \geq \theta$ が成立することをいう．アイテムセット P がバスケット対応密集区間を少なくとも *1* つもつとき，P を バスケット対応密集パターン（*basket-aware dense pattern*）と呼ぶ．

定義 5 (偽共起パターン)**.** データ全体（$I = [ts_1, ts_N]$）におけるアイテムセット P が 偽共起パターン（*spurious pattern*）であるとは，ある最小サポートしきい値 θ に対して

$$txn_support(P) \geq \theta \quad \text{かつ} \quad true_support(P) < \theta \tag{4}$$

が成立することをいう．

評価指標 (SPR): 最小サポート θ に対して，txn 頻出パターン集合を $\mathcal{F}_{\mathrm{txn}} = \{P \mid |P| \geq 2,\ \mathrm{txn_support}(P) \geq \theta\}$，そのうち偽共起パターン集合を $\mathcal{F}_{\mathrm{spur}} = \{P \in \mathcal{F}_{\mathrm{txn}} \mid \mathrm{true_support}(P) < \theta\}$ とする．偽共起パターン率（SPR: Spurious Pattern Rate）を

$$\mathrm{SPR} = \frac{|\mathcal{F}_{\mathrm{spur}}|}{|\mathcal{F}_{\mathrm{txn}}|} \tag{5}$$

と定義する．$\mathrm{SPR} = 0$ は偽共起パターンが存在しないことを，$\mathrm{SPR} = 1$ は txn 頻出パターンがすべて偽共起であることを意味する．

注記: 式 (5) はデータ全体のグローバルサポートに基づく定義であり，局所密集パターンの偽共起率の評価は今後の課題とする．

D. 問題の定式化

以上の定義に基づき，本稿が解く問題を次のように定式化する．

問題 (Basket-aware Dense Pattern Mining): 入力として，バスケット構造付きトランザクションデータ D，最小サポート θ，最小区間長 W，最大アイテムセット長 L を与える．出力として，すべてのバスケット対応密集パターン P（$2 \leq |P| \leq L$）とその密集区間の集合を網羅的に列挙する．

従来法との関係: トランザクション内サポート (式 (2)) を用いた密集パターン抽出が従来法（flatten 後の Apriori-window）に対応する．提案手法はこれをバスケット内サポート（式 (1)）に置き換えることで偽共起パターン（定義 5）を出力から完全に排除する．

命題 2 (提案手法の SPR 保証)**.** 提案手法（*Phase 1*）が出力するすべての密集パターンについて，*SPR = 0* が成立する．

Proof. 提案手法はバスケット内サポート（true_support）が θ 以上のパターンのみを出力する．偽共起パターン（定義 5）は $\text{true_support}(P) < \theta$ であるため，提案手法の出力に含まれない．したがって，出力された txn 頻出パターンに占める偽共起パターンの割合は 0 であり，$\text{SPR} = 0$ が成立する． □

IV. 提案手法: Basket-aware Apriori-Window

A. 従来法（*flatten*）の問題

従来の Apriori-window [1] はバスケット構造を扱う設計になっていないため，バスケット構造付きトランザクションデータに適用する際には，各トランザクションの全バスケットを統合（flatten）して単一アイテムセットに変換する前処理が必要である．

定義 6 (フラット化)**.** バスケット構造付きトランザクション $d = (ts, \mathcal{B})$ の フラット化（*flatten*）とは，全バスケットの和集合を計算し，$d_{flat} = (ts, \bigcup_{B \in \mathcal{B}} B)$ という単一アイテムセットのトランザクションを生成することである．

フラット化後のトランザクションデータに Apriori-window を適用することで，トランザクション内サポート（式 (2)）に基づく密集パターンが抽出される．このとき，命題 1 より $\text{txn_support}(P) \geq \text{true_support}(P)$ であるため，従来法は真の密集パターンに加えて偽共起パターンも検出してしまう（定義 5）．

図 3 は，従来法がバスケットを統合してトランザクション内共起を数えるのに対し，提案法がバスケット境界を維持して共起を評価する差分を示している．

B. アルゴリズム概要

提案手法 *Basket-aware Apriori-Window*（Phase 1）は，バスケット内共起（定義 2）のみに基づいてサポートを計算することで，原理的に偽共起パターンを出力しない．アルゴリズム 1 に全体の手順を示す．

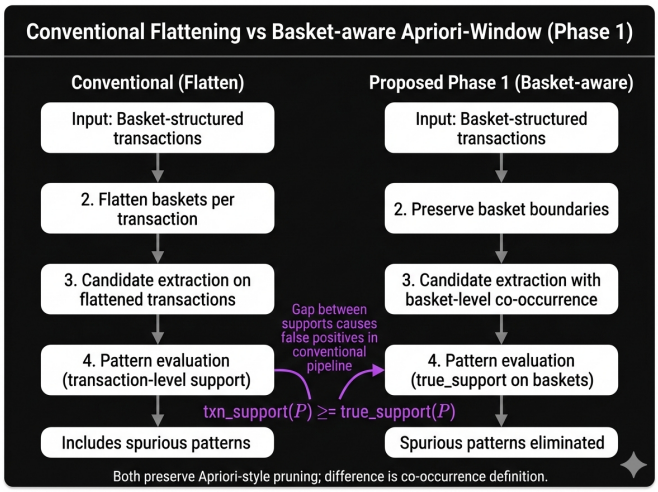

Fig. 3: 従来法（flatten）と提案手法（Phase 1）の処理比較

1) ステップ 1: バスケットマップの構築: BuildBasketMap は，データ全体を 1 回走査して 3 つのデータ構造を構築する．

- $M_{\text{item}}[i]$（アイテム → バスケット ID マップ）: アイテム i が含まれるバスケットのグローバル ID のリスト（昇順ソート済み）．同一バスケット内に同じアイテムが複数あっても 1 回のみ記録する．
- $\text{b2t}[b]$（バスケット → トランザクション ID マップ）: バスケット b が属するトランザクションのインデックス．トランザクション順 × バスケット順でグローバル連番を付与するため，この配列は単調非減少となる．
- $M_{\text{txn}}[i]$（アイテム → トランザクション ID マップ）: アイテム i が（いずれかのバスケットに）含まれるトランザクションのインデックスのリスト（重複なし・昇順）．長さ 1 パターンのスライディングウィンドウ処理で使用する．

2) ステップ 2: 長さ 1 の密集パターン: 各単体アイテム i に対して，$M_{\text{txn}}[i]$（アイテムの出現トランザクション ID 列）を入力としてスライディングウィンドウ機構を実行し，$\text{true_support}_I(i) \geq \theta$ かつ $n(D_I) \geq W$ を満たすすべての密集区間を求める．長さ 1 のアイテムについては，バスケット内共起とトランザクション内共起が一致する（1 アイテムは常に単一バスケット内）ため，この処理は従来法と等価である．

3) ステップ 3: 長さ $k \geq 2$ の密集パターン（核心部）: アルゴリズム 1 の 21〜27 行目が本手法の核心部である．候補アイテムセット P に対して，バスケット ID の集合積 $B_P = \bigcap_{i \in P} M_{\text{item}}[i]$ を計算する．B_P は P のすべてのアイテムを同時に含むバスケットの集合であり，バスケット内共起を直接エンコードする．

続いて，B_P を b2t を用いてトランザクション ID 列 T_P に変換し（BasketToTransaction），スライディングウィンドウを適用して密集区間を求める．

従来法との違い: 従来法では，候補 P のサポートを計

Algorithm 1 Basket-aware Apriori-Window

Require: バスケット構造付きトランザクションデータ D，最小サポート θ，最小区間長 W，最大長 L
Ensure: バスケット対応密集パターンとその密集区間の集合 $\mathcal{F}$

$\mathcal{F} \leftarrow \emptyset$
▷ — ステップ 1: バスケットマップの構築 —
$(M_{\text{item}}, \text{b2t}, M_{\text{txn}}) \leftarrow \textsc{BuildBasketMap}(D)$
▷ $M_{\text{item}}[i]$: アイテム i が含まれるバスケット ID のリスト
▷ $\text{b2t}[b]$: バスケット b が属するトランザクション ID
▷ $M_{\text{txn}}[i]$: アイテム i が含まれるトランザクション ID のリスト
▷ — ステップ 2: 長さ 1 の密集パターン —
$\mathcal{F}_1 \leftarrow \emptyset$
for each item $i \in \mathcal{I}$ **do**
  intervals $\leftarrow \textsc{SlidingWindow}(M_{\text{txn}}[i], \theta, W)$
  if intervals $\neq \emptyset$ **then**
    $\mathcal{F}_1 \leftarrow \mathcal{F}_1 \cup \{(\{i\}, \text{intervals})\}$
  end if
end for
$\mathcal{F} \leftarrow \mathcal{F} \cup \mathcal{F}_1$
▷ — ステップ 3: 長さ $k \geq 2$ の密集パターン —
$k \leftarrow 2$
while $\mathcal{F}_{k-1} \neq \emptyset$ **and** $k \leq L$ **do**
  $\mathcal{C}_k \leftarrow \textsc{GenerateCandidates}(\mathcal{F}_{k-1})$
    ▷ Apriori 反単調性による枝刈り済み候補集合
  $\mathcal{F}_k \leftarrow \emptyset$
  for each candidate $P \in \mathcal{C}_k$ **do**
    ▷ **Key: バスケットレベルの共起集合を計算**
    $B_P \leftarrow \bigcap_{i \in P} M_{\text{item}}[i]$
    ▷ B_P: P の全アイテムを同時に含むバスケット ID の集合
    $T_P \leftarrow \textsc{BasketToTransaction}(B_P, \text{b2t})$
    ▷ T_P: B_P に対応するトランザクション ID のリスト
    intervals $\leftarrow \textsc{SlidingWindow}(T_P, \theta, W)$
    if intervals $\neq \emptyset$ **then**
      $\mathcal{F}_k \leftarrow \mathcal{F}_k \cup \{(P, \text{intervals})\}$
    end if
  end for
  $\mathcal{F} \leftarrow \mathcal{F} \cup \mathcal{F}_k$
  $k \leftarrow k + 1$
end while
return $\mathcal{F}$

算するために $T_P = \bigcap_{i \in P} M_{\text{txn}}[i]$ というトランザクション ID の集合積を用いる．これは，各アイテムがそのトランザクションのいずれかのバスケットに存在することを確認するだけで，同一バスケット内の共起は保証しない．提案手法は集合積をバスケット粒度で行うことで，この問題を根本的に解決する．

4) スライディングウィンドウ機構*:* SlidingWindow の実装は先行研究 Apriori-window [1] と同一である．固定幅 W のウィンドウを 1 トランザクション単位でスライドさせながら，ウィンドウ内のサポートが θ を超える区間を記録し，隣接区間を統合して密集区間リストを返す．また，ウィンドウ内でサポートが θ を大幅に上回る場合には，余剰分に基づいてストライドを増加させる最適化（stride adjustment）を適用して効率的な走査を実現する [1].

C. 反単調性の証明

Apriori の枝刈りが機能するためには，候補のサポートが単調非増加（反単調性）であることが必要である．従来の Apriori-window では定義域をトランザクション単位の出現集合としてこれを証明しているが [1]，提案手法ではバスケット粒度での反単調性を証明する必要がある．

定理 1 (バスケット内共起における反単調性)**.** 任意のアイテムセット P とその真の上位集合 $P' \supset P$，任意の区間 I に対して，

$$\mathit{true_support}_I(P') \leq \mathit{true_support}_I(P) \tag{6}$$

が成立する．

Proof. $d = (ts, \mathcal{B})$ を区間 I 内の任意のトランザクションとする．

P' がバスケット内共起する（$\exists B \in \mathcal{B} : P' \subseteq B$）ならば，$P \subset P'$ より $P \subseteq B$ も成立するため，P もそのバスケット B でバスケット内共起する．

逆は一般に成立しない（P がバスケット内共起しても，P' の余分なアイテムが同一バスケットに含まれるとは限らない）．

したがって，P' がバスケット内共起するトランザクションの集合は，P がバスケット内共起するトランザクションの集合の部分集合であり，式 (6) が成立する． □

定理 1 より，あるアイテムセット P がバスケット対応密集パターンでない（すなわち，すべての区間 I で $\text{true_support}_I(P) < \theta$ または $n(D_I) < W$）ならば，P の任意の上位集合 P' もバスケット対応密集パターンにならない．これにより，GenerateCandidates（$\mathcal{F}_{k-1}$ に含まれないパターンを部分集合にもつ候補を除外する Apriori 枝刈り）の正当性が保証される．

D. 従来 *Apriori-window* との関係

命題 3 (特殊ケースとしての従来法)**.** 各トランザクションがちょうど *1* つのバスケット（$|\mathcal{B}| = 1$）をもつとき，提案手法（*Phase 1*）の出力は従来 *Apriori-window* の出力と一致する．

Proof. $|\mathcal{B}| = 1$ の場合，バスケット内共起とトランザクション内共起は同値であるため（$\exists B \in \mathcal{B} : P \subseteq B \Leftrightarrow P \subseteq \bigcup_{B \in \mathcal{B}} B$），$\text{true_support}_I(P) = \text{txn_support}_I(P)$ が成立する．したがって，提案手法と従来法は同一の密集パターンを出力する． □

命題 3 は，提案手法が従来 Apriori-window の 上位互換（*strict generalization*）であることを示す．すなわち，従来法は $\lambda_{\text{baskets}} = 1$（各トランザクションにバスケットが 1 つしかない場合）の特殊ケースとして包含される．

E. 実装上のポイント

1) BasketToTransaction におけるトランザクション *ID* の重複*:* BasketToTransaction は，バスケット ID リスト B_P を対応するトランザクション ID 列 T_P に変換する．ここで，B_P に同一トランザクションの複数バスケット b_1, b_2 が含まれる（$\mathrm{b2t}[b_1] = \mathrm{b2t}[b_2] = t$）場合，トランザクション t は T_P に *1* 回のみ記録される設計とする．

これは，true_support の定義（式 (1)）がトランザクション単位でカウントを行うことに対応している．すなわち，1 つのトランザクション内に P を含むバスケットが複数あっても，サポートへの貢献は 1 とみなす．

2) グローバルバスケット ID の採番: バスケット ID はトランザクション順 × バスケット順にグローバル連番を付与する．この採番規則により，b2t 配列が単調非減少となり，ソート済みリストとの二分探索による高速な集合積計算（バスケット ID の積集合）が $O(|B_{P_1}|+|B_{P_2}|)$（$P = \{i_1, i_2\}$ の場合のマージ）で可能になる．

3) メモリ効率: 長さ 1 の密集パターンのみに関与する単体アイテムについては，$M_{\mathrm{item}}[i]$ は長さ 1 パターンの密集区間計算後に解放可能である．ただし，長さ 2 以上の候補生成時には再び必要になるため，全プロセスを通じて保持する実装が現実的である．

V. 実験結果と考察

A. 実験方法

本研究の評価は，合成データ上で提案手法（Phase 1）と従来法（flatten + Apriori-window）を比較する構成で行った．再現性確保のため，本文では実験条件の要点を示し，実行コマンドと出力ファイル対応は補遺（Sec. A-A）に集約する．

a) データ生成: 各トランザクション t について，まずバスケット数 n_t を $\max(1, \mathrm{Poisson}(\lambda_{\mathrm{baskets}}))$ で生成する．次に各バスケット b のサイズ k_b を $\max(1, \mathrm{Poisson}(\lambda_{\mathrm{basket_size}}))$ で生成し，主カテゴリを G 個から一様に選択する．各アイテムは確率 p_{same} で主カテゴリ内，確率 $1 - p_{\mathrm{same}}$ で全体語彙から Zipf 重み（指数 α_{zipf}）に従ってサンプルし，重複除去後の集合をバスケットとする．同一トランザクション内の異なるバスケット間には真の共起関係を仮定しないため，フラット化で偽共起が発生する条件を制御できる．

b) 比較条件: 提案手法はバスケット境界を保持して support を計算する．従来法は各トランザクション内の全バスケットを統合して単一アイテム集合として扱う．共通パラメータは，$min_support = 50$（件数しきい値），$window_size = 500$，$max_length = 4$ とした．

c) 評価指標: 偽共起の強さは GT SPR と observed spurious で評価した．効率は，Phase 1 の core 時間と従来法の total 時間（flatten + core）で比較した．各条件は複数 seed で実行し，表には平均値を報告する．

d) 実装: 実験は乱数シードを固定したスクリプト群によりオーケストレーションし，再現性を確保した．A1/A2 は 5 seeds，A4 は 3 seeds で実行し，表には各条件の平均値を報告する．実行手順・環境・出力ファイルの詳細は補遺（Sec. A-A）に記載する．

TABLE I: 再現用の固定パラメータ（Stage A 共通）

項目	値
N_{items}	200
$\lambda_{\mathrm{basket_size}}$	3.0
p_{same}	0.8
α_{zipf}	1.2
min_support	50
window_size	500
max_length	4

TABLE II: A1 結果（平均）

$\lambda_{\mathrm{baskets}}$	GT SPR	Obs. Spur.	Trad./P1 Multi	Trad./P1 Time
1.0	0.454	1.0	2.00	1.02
1.5	0.676	3.8	3.11	1.08
2.0	0.794	10.2	3.50	1.06
3.0	0.906	50.4	4.41	1.49
5.0	0.971	379.0	12.28	3.05

B. 設定

本節では Stage A のうち，以下の 3 実験を報告する．

- A1: $\lambda_{\mathrm{baskets}} \in \{1.0, 1.5, 2.0, 3.0, 5.0\}$ の sweep(seed=5)
- A2: カテゴリ数 $G \in \{3, 5, 10, 20, 50\}$ の sweep (seed=5)
- A4: $N_{\mathrm{transactions}} \in \{10^3, 10^4, 10^5, 10^6\}$ の sweep (seed=3)

主要指標は，偽共起率（GT SPR），従来法のみが検出した多項目パターン数（observed spurious），および実行時間（Phase 1 core vs 従来法 total）である．

C. A1: バスケット数増加に対する偽共起の拡大

表 II に A1 の平均結果を示す．$\lambda_{\mathrm{baskets}}$ の増加に伴い，データ生成側の GT SPR は 0.454 から 0.971 まで単調増加した．同時に observed spurious は 1.0 から 379.0 へ増加し，従来法の過検出が急激に拡大した．

実行時間面では，低 λ では差が小さい一方，高 λ (=5.0) では従来法が 3.05 倍遅くなった．これは，フラット化により候補が増え，従来法の探索空間が拡大したためと解釈できる．

D. A2: カテゴリ数増加による構造的偽共起の低下

表 III に A2 の平均結果を示す．カテゴリ数 G を増やすと，同一カテゴリ内での偶発的共起が起こりにくくなるため，GT SPR は 0.827（$G = 3$）から 0.638（$G = 50$）へ低下した．observed spurious も 24.4 から 3.0 に減少し，偽共起がデータ構造に依存して変動することを確認できる．

E. A4: スケーラビリティ

表 IV に A4 の平均結果を示す．10^3 から 10^6 トランザクションまで，Phase 1・従来法ともに計算時間は増加するが，従来法の増加率がより大きい．特に 10^6 では，Phase 1 が 95.5 秒，従来法が 143.6 秒で，従来法は 1.50 倍遅い．

以上より，提案法は偽共起抑制だけでなく，高密度・大規模条件での実行効率にも優位性を示した．

TABLE III: A2 結果（平均）

G	GT SPR	Obs. Spur.	Trad./P1 Multi
3	0.827	24.4	3.14
5	0.822	19.0	2.83
10	0.794	10.2	3.50
20	0.722	5.8	4.22
50	0.638	3.0	2.50

TABLE IV: A4 結果（平均実行時間, ms）

$N_{\text{transactions}}$	Phase 1	Traditional	Trad./P1
10^3	10.41	11.76	1.13
10^4	139.34	158.50	1.14
10^5	2488.37	3137.22	1.26
10^6	95489.56	143625.49	1.50

VI. 結論

本稿は，Apriori-window の枠組みを維持しつつ，入力データのバスケット構造を明示的に扱う Basket-aware Apriori-Window（Phase 1）を提案した．提案法は，共起判定をトランザクション内からバスケット内へ置き換えることで，偽共起の原因となるバスケット横断混入を抑制する．

合成データ実験では，(i) バスケット数増加時に従来法の偽共起が急増すること（A1），(ii) カテゴリ構造の変化に応じて偽共起率が変動すること（A2），(iii) 大規模条件で提案法の実行時間優位が拡大すること（A4）を確認した．特に 10^6 トランザクションでは，提案法 95.5 秒，従来法 143.6 秒であり，従来法は 1.50 倍遅かった．

今後の課題としては，(1) 実データ（Stage B）での外的妥当性評価，(2) 区間品質指標（Purity/Jaccard）を含む総合評価，(3) 並列化・メモリ管理の最適化によるさらなる高速化が挙げられる．

Appendix A
Supplementary Material

A. 再現実行情報

本稿の Stage A（A1/A2/A4）結果は，以下の手順で再現可能である．再現対象は 2026-03-02 実行の成果物である．

a) 前提環境:

- OS: macOS / Linux（検証は macOS）
- 言語処理系: Python 3 系
- リポジトリルート: 各自のチェックアウト先（以下 `<repo>` と表記）

b) 実行入口:

- Phase 1 / 従来法の Stage A 実験: `experiments/run_stage_A.py`
- 生成器（単体実行時）: `experiments/gen_synthetic.py`
- 比較実行器（単体実行時）: `experiments/run_phase1.py`
- 比較手法 runner: `baselines/runner/run_stage_a_suite.py`
- 比較手法 Rust バイナリ: `apriori_window_suite/src/bin/comparative_mining.rs`

c) Stage A 一括実行コマンド: 以下は本稿と同じディレクトリ構成で出力を得るための最小コマンドである．

```
cd <repo>

# A1（lambda sweep）
python3 experiments/run_stage_A.py \
  --experiments A1 \
  --output-dir experiments/results/a1_full_20260302 \
  --data-dir experiments/results/a1_full_20260302_data

# A2（G sweep）
python3 experiments/run_stage_A.py \
  --experiments A2 \
  --output-dir experiments/results/a2_full_20260302 \
  --data-dir experiments/results/a2_full_20260302_data

# A4（実装上は A3: N sweep）
python3 experiments/run_stage_A.py \
  --experiments A3 \
  --output-dir experiments/results/a3_full_20260302 \
  --data-dir experiments/results/a3_full_20260302_data
```

d) A4 集計ファイルへの変換: 本文ではスケーラビリティ実験を A4 と呼称するため，`a3_full_20260302/A3_timing.csv` を `a4_full_20260302/A4_scalability.csv` として再配置し，平均表 `A4_scalability_summary.csv` を併せて用いる．この名称対応のみで，行データ自体は同一である．

B. A1/A2/A4 の出力ファイル

本文で報告した結果は，以下の CSV に保存される．

- A1: `experiments/results/a1_full_20260302/A1_spr.csv`
- A2: `experiments/results/a2_full_20260302/A2_spr.csv`
- A4: `experiments/results/a4_full_20260302/A4_scalabilit`

各 CSV と本文表の対応は以下である．

- Table II: `A1_spr.csv` を λ_{baskets} ごとに seed 平均
- Table III: `A2_spr.csv` を G ごとに seed 平均
- Table IV: `A4_scalability_summary.csv` の平均値列

C. データ形式と乱数設定

生成データ（`*_seedX.txt`）は 1 行 1 トランザクション形式で，同一行内のバスケットを区切り文字 `" | "` で連結する．例: `1 4 9 | 2 8`

乱数は Python の `random.Random(seed)` を用い，seed は A1/A2 で `0..4`，A4 で `0..2` を使用する．したがって，同一 seed・同一パラメータ・同一 Python 実行系でトランザクションファイルは決定的に再生成される．

D. Ground Truth の再計算

GT 判定は `experiments/gen_synthetic.py` の `compute_ground_truth()` を用いて実施する．多項目集合 S（$2 \le |S| \le max_length$）に対して，

- `true_support(S)`: 同一バスケット内で S が成立した回数

- `txn_support(S)`: トランザクション単位（バスケット横断可）で S が成立した回数

を計数し，`true_support(S) < min_support <= txn_support(S)` を満たす場合を spurious と判定する．SPR は本文 Eq. (5) に対応し，分母を `txn_support >= min_support` の多項目集合数とした比率として計算する．

E. 実験マッピングに関する注記

現行スクリプトでは，`run_stage_A.py` の実験名と設計書上の表記が一部異なる．具体的には，スクリプト内 `A3` が設計書上のスケーラビリティ実験（A4）に対応する．本稿では設計書表記（A4）に合わせて結果を報告した．

F. 再現時の確認項目

再実行後の整合確認は，以下の最小チェックを推奨する．

- A1: λ_{baskets} 増加に伴い GT SPR が単調増加すること
- A2: G 増加に伴い GT SPR が減少傾向を示すこと
- A4: 従来法時間 / Phase 1 時間が $N = 10^6$ で約 1.50 になること

References

[1] （著者名）, "Data mining methods based on swarm intelligence to extract dense itemsets," in *Proceedings of the IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, 2025, （投稿中 / published 後に更新）.

[2] R. Agrawal, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, 1994, pp. 487–499.

[3] J. Han, J. Pei, and Y. Yin, "Mining frequent patterns without candidate generation," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 1–12, 2000.

[4] M. J. Zaki, S. Parthasarathy, M. Ogihara, W. Li *et al.*, "New algorithms for fast discovery of association rules," in *Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining*, vol. 97, 1997, pp. 283–286.

[5] P. Fournier-Viger, J. C.-W. Lin, B. Vo, T. T. Chi, J. Zhang, and H. B. Le, "A survey of itemset mining," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 4, p. e1207, 2017.

[6] S. K. Tanbeer, C. F. Ahmed, B.-S. Jeong, and Y.-K. Lee, "Discovering periodic-frequent patterns in transactional databases," in *Proceedings of the 13th Pacific-Asia Conference on Knowledge Discovery and Data Mining*, 2009, pp. 242–253.

[7] R. U. Kiran, J. Venkatesh, M. Toyoda, M. Kitsuregawa, and P. K. Reddy, "Discovering partial periodic-frequent patterns in a transactional database," *Journal of Systems and Software*, vol. 125, no. C, pp. 170–182, 2017.

[8] A. K. Mahanta, F. A. Mazarbhuiya, and H. K. Baruah, "Finding locally and periodically frequent sets and periodic association rules," in *Proceedings of the 1st International Conference on Pattern Recognition and Machine Intelligence*, 2005, pp. 576–582.

[9] P. Fournier-Viger, P. Yang, R. U. Kiran, S. Ventura, and J. M. Luna, "Mining local periodic patterns in a discrete sequence," *Information Sciences*, vol. 544, pp. 519–548, 2021.